

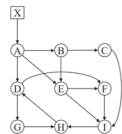


复习课（五） （运行时环境）

冯洋
南京大学计算机学院
fengyang@nju.edu.cn



七、简答题（共1题，共10分）
现有如下图所示的对象网络：

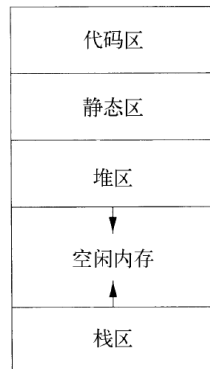


1. 从X指向A的指针被删除时，对象的引用计数会发生哪些变化，画出对应的指针被删除后的对象网络（保留下来的对象请在字母外画上实线圈，被删除对象不作处理，保留的指针画上箭头，被删除的指针不作处理）。



存储分配的典型方式

- 静态区、堆区、栈区分别放置不同类型生命期的数据值
 - 静态区
 - 程序的某些数据对象的大小可以在编译时刻知道，可被放置在另一个称为静态区的区域
 - 该区域可被静态确定
 - 放置的数据对象包括全局常量和编译器产生的数据，如用于支持垃圾回收的信息等



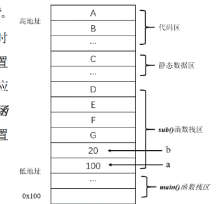
- 请考虑以下C语言代码。我们假设BP是指向栈底的指针，SP是指向栈顶的指针，C语言栈帧的结构如右图所示。本题中，栈帧布局地址增长遵循课本规定，栈是由低地址向高地址增长，其他部分则是由高地址向低地址增长。

- (1) A: _____
- (2) B: _____
- (3) C: _____
- (4) D: _____
- (5) E: _____
- (6) F: _____
- (7) G: _____
- (8) SP: _____
- (9) BP: _____

```
(1) int sub(int a, int b);
(2) int m = 0;
(3) void main(){
(4) int i = 100;
(5) int j = 20;
(6) m = sub(i, j);
(7) printf("m=%d", m);
(8) }
(9)
(10) int sub(int a, int b){
(11) int res = 0;
(12) res = a - b;
(13) return res;
(14) }
```



假设程序运行完第12行，即res计算完毕时。请在如下栈帧结构图中，分别写出此时A.B.C.D.E.F.G对应位置的内容，若对应位置可以推断具体值（值可以为null），请写明对应变量名及其值，并在图中通过箭头表明sub()函数的SP和BP的位置（假设SP和BP的位置在res被定义时已经确定）。

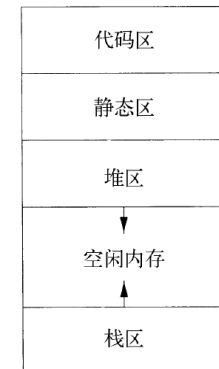


- 2. 从A指向B的指针被删除时，引用计数会发生哪些变化，画出对应的指针被删除后的对象网络。



存储分配的典型方式

- 静态区、堆区、栈区分别放置不同类型生命期的数据值
 - 栈和堆
 - 为了将运行时刻的空间利用率最大化，栈和堆被放在剩余地址空间的相对两端
 - 这两个区域是动态的，根据需要会向对方增长
 - 栈区用来存放称为活动记录的数据结构，这些活动记录在函数调用过程中生成
 - 实践中，栈向较低地址方向增长，堆向较高地址方向增长
 - 我们假定栈是由低地址向高地址增长，以便我们能在示例中方便地使用正的偏移量



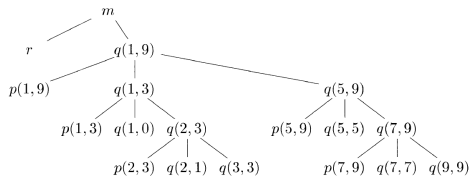


活动树的例子 (1)

■ 程序

○ 活动树与程序行为对应关系

- 过程调用的序列和活动树的前序遍历相对应。
- 过程返回的序列和活动树的后序遍历相对应。



```
enter main()
  enter readArray()
  leave readArray()
  enter quicksort(1,9)
    enter partition(1,9)
    leave partition(1,9)
    enter quicksort(1,3)
    ...
  leave quicksort(1,3)
  enter quicksort(5,9)
  ...
  leave quicksort(5,9)
  leave quicksort(1,9)
leave main()
```



调用/返回代码序列的要求

■ 数据方面

- 能够把参数正确地传递给被调用者
- 能够把返回值传递给调用者

■ 控制方面

- 能够正确转到被调用过程的代码开始位置
- 能够正确转回调用者的调用位置（的下一条指令）

- 调用代码序列和活动记录的布局相关



活动记录

■ 过程调用和返回由控制栈进行管理

- 每个活跃的活动对应于栈中的一个活动记录

■ 活动记录按照活动的开始时间, 从栈底到栈顶排列

- 活动树的根位于栈底
- 栈中全部活动记录的序列对应于在活动树中到达当前控制所在的活动结点的路径

实在参数
返回值
控制链
访问链
保存的机器状态
局部数据
临时变量



一个简单的例子

■ 两个函数

■ 一个全局变量m

■ 若干局部变量i, j, c, a...

■ 一次函数调用

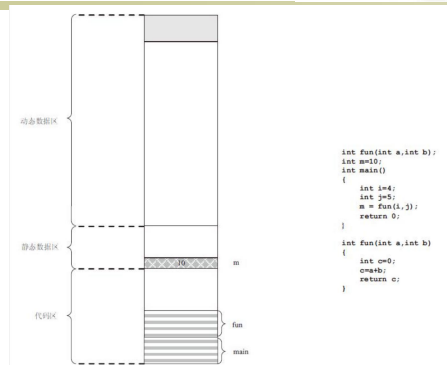
```
int fun(int a, int b);
int m=10;
int main()
{
    int i = 4;
    int j = 5;
    m = fun(i,j);
    return 0;
}

int fun(int a, int b)
{
    int c = 0;
    c = a + b;
    return c;
}
```



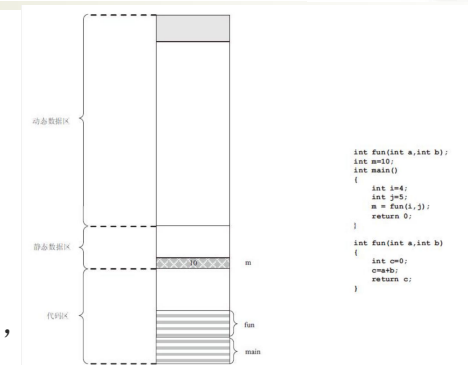
一个简单的例子

- 整个程序放到了内存中的样子如右图。共有三个区域：代码区，静态数据区，动态数据区



一个简单的例子

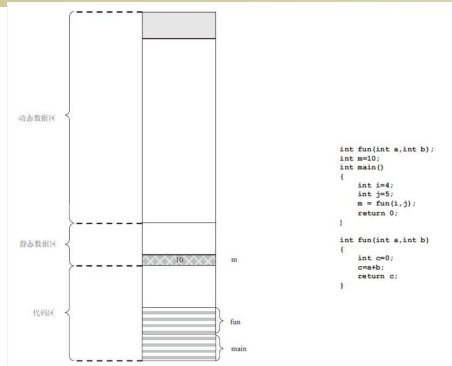
- 静态数据区：可以在编译时刻知道大小的数据对象，包括全局常量和编译器产生的数据（比如支持垃圾回收的信息等）
- 动态数据区：无法在编译时刻知道大小的数据对象
- 代码区：因为目标代码在编译时刻已经知道具体的大小，代码区也是一种静态区





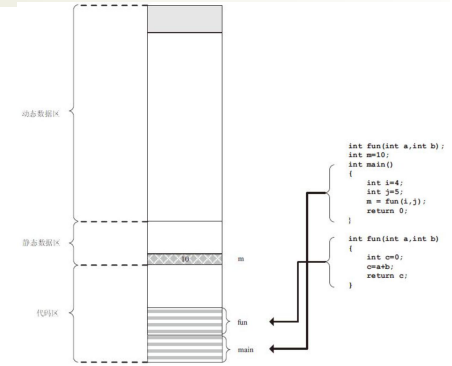
一个简单的例子

- 静态数据区：可以在编译时刻知道大小的数据对象，包括全局常量和编译器产生的数据（比如支持垃圾回收的信息等）
- 动态数据区：无法在编译时刻知道大小的数据对象
- 代码区：因为目标代码在编译时刻已经知道具体的大小，代码区也是一种静态区



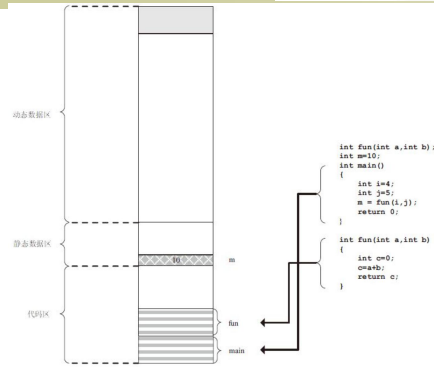
一个简单的例子

- 全局变量m的数值装载在静态数据区中
- 函数fun和main的代码装载在代码区中
- 程序开始执行前，动态数据区中没有数据



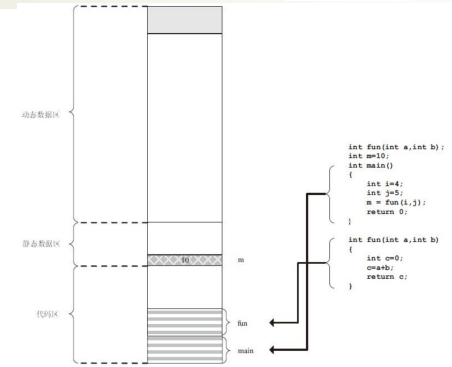
一个简单的例子

- 全局变量m的数值装载在静态数据区中
- 函数fun和main的代码装载在代码区中
- 程序开始执行前，动态数据区中没有数据；只有在程序开始执行后，在指令的驱动下，这一区域才会产生数据，压栈和清栈的工作就是在这一区域完成的



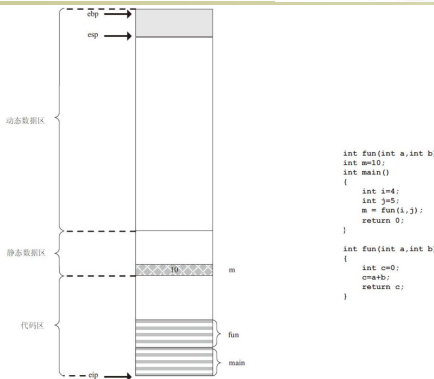
一个简单的例子

- 程序执行的本质，就是代码区的指令不断执行
 - 驱使动态数据区和静态数据区产生数据变化



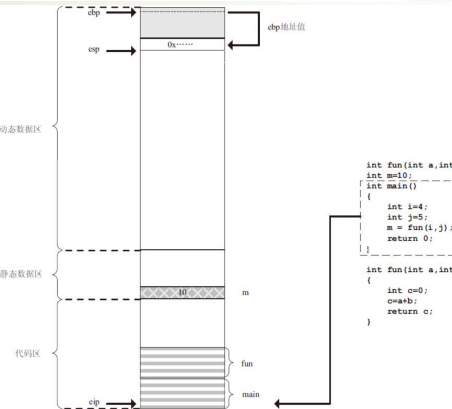
一个简单的例子

- 我们假定CPU中有三个寄存器，分别是eip、ebp、esp。其中eip永远指向代码区要执行的下一条指令
 - ebp与esp的位置在程序加载时由内核设置
 - eip的位置一开始指向main函数
- ebp和esp用来管理栈空间
 - ebp指向栈底，esp指向栈顶，代码区中的程序执行，函数调用、返回，均存在不断的压栈与清栈
 - 栈底画在栈顶的上方



一个简单的例子

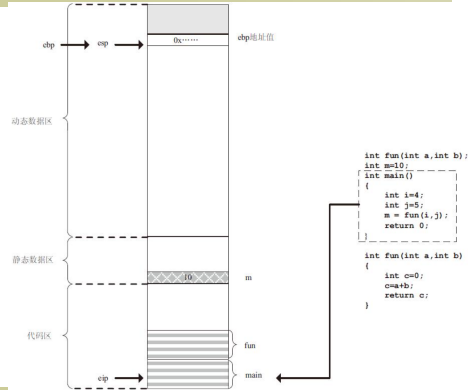
- 程序开始执行main函数第一条指令，eip自动指向下一条指令。第一条指令的执行，则使得ebp的地址值被保存到栈里面
- 保存的目的是本程序执行完毕后，ebp还能返回到现在的位置，复原现在的栈。ebp保存之后，esp则自动向栈顶移动，它永远指向栈顶





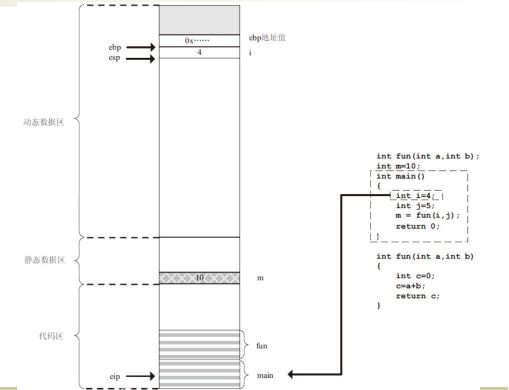
一个简单的例子

- 在对返回地址保存之后，开始为main构建栈，main的栈构建情况如右图所示
- 注意，此事ebp原来指向的值已经被存起来了，所以esp就自动向栈顶移动，同时，ebp需要用来指向main的栈底
- 此时由于没有数据在栈中，ebp和esp的位置其实是重叠的



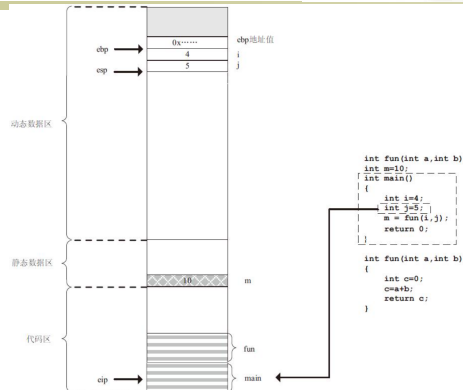
一个简单的例子

- 程序继续执行，第一个语句，int i = 4 被执行
- 注意到，对main来说，i 是一个局部变量，那么，其初始值4，被压入栈中
- 特别注意栈顶指针esp的变化



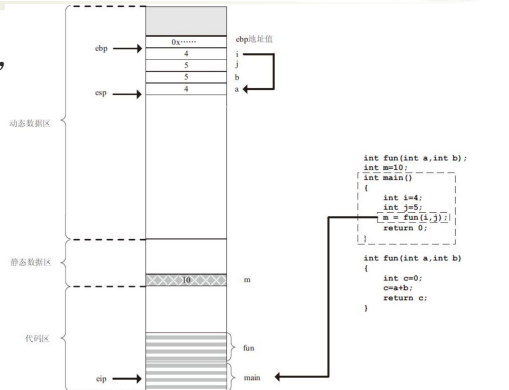
一个简单的例子

- 同样的道理，int j = 5 被执行，5 被压入到栈



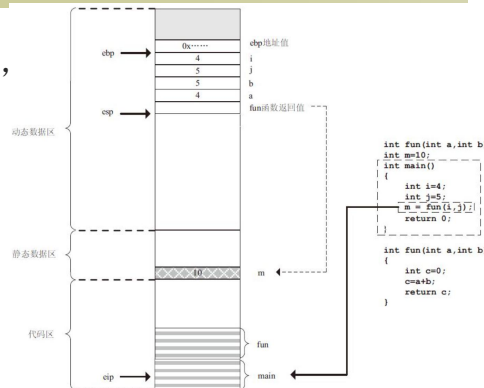
一个简单的例子

- 下一步，程序执行到调用函数fun的指令。特别注意，此处，我们对fun的调用，首先需要完成的，是传参指令
- 特别注意，在C语言中，参数入栈顺序与传参书写顺序正好相反。参数b先入栈，b的数值是局部变量j的数值5。因此，情况如右所示



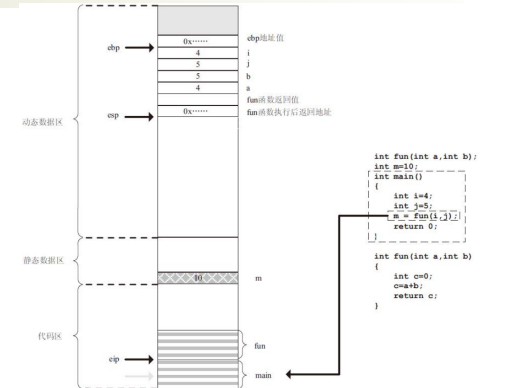
一个简单的例子

- 程序继续执行，那么应该压入的是fun函数的返回值，后面返回值应该是传递给m



一个简单的例子

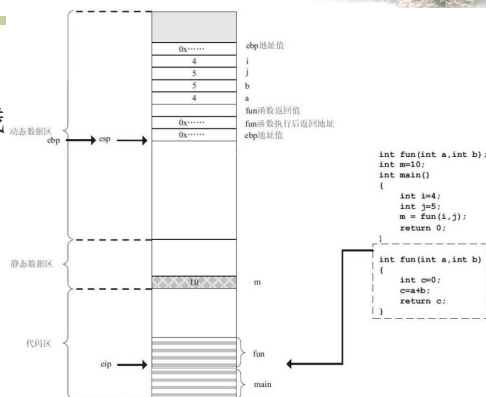
- 在保存好fun函数执行完毕的返回地址后，eip跳转到fun的代码区进行执行，如右图所示
- 后续则需要构建fun的栈





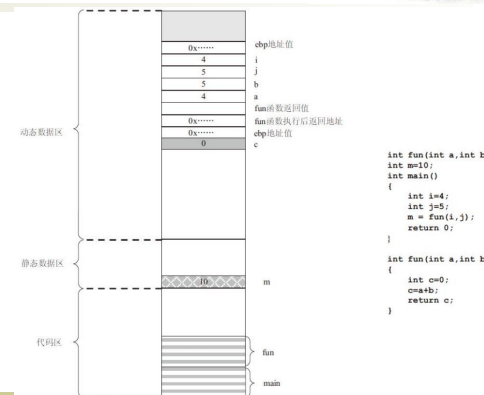
一个简单的例子

- 与前面我们调用main函数相似，ebp与esp再次指向同一个位置，此时fun的栈是空的



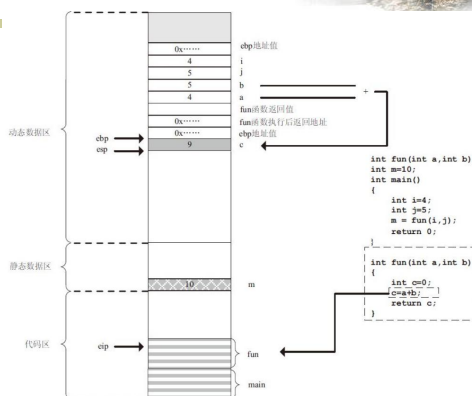
一个简单的例子

- 与前面的情况类似，c会被压入fun的栈中
- 后续是若干运算指令



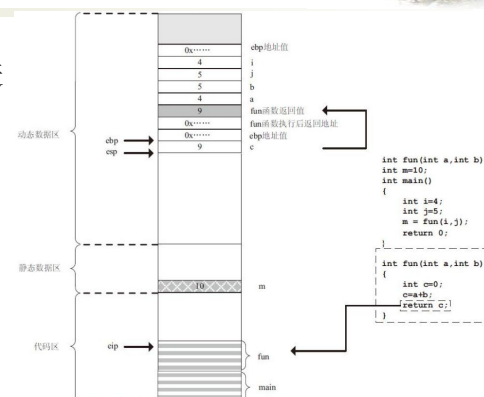
一个简单的例子

- 在执行a+b的过程中，我们可以看到，在运算指令的执行过程中，可以通过一系列的偏移计算，得到运算指令需要的数值
- 运算指令执行完毕后，赋值给局部变量c



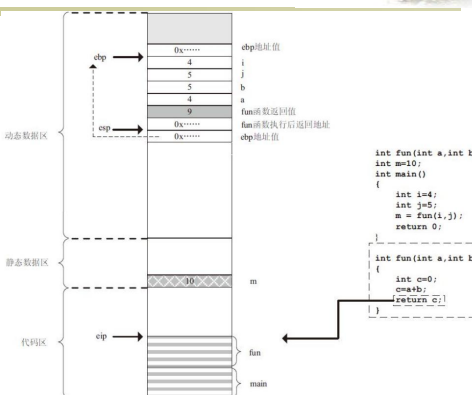
一个简单的例子

- 局部变量c的值作为返回值，对fun函数的返回值进行赋值



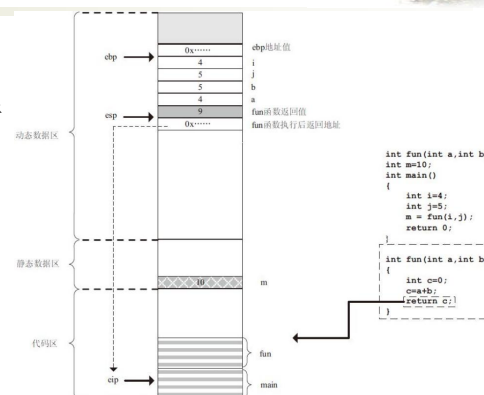
一个简单的例子

- fun函数执行完毕，要恢复main函数调用fun函数的现场
- 这一现场包括两个部分
 - 一部分是main函数的栈要恢复，包括栈顶和栈底
 - 另一部分是要找到fun函数执行后的返回地址，然后再跳转到那里去继续执行
 - ebp的恢复相对简单，之前在此处有存储ebp的地址值，在消栈操作中，只需要取出该值赋值给ebp指针即可



一个简单的例子

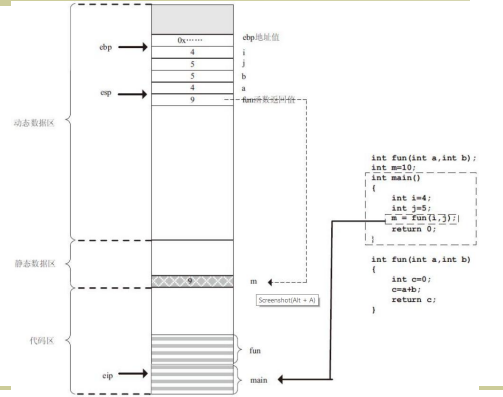
- 在ebp恢复后，esp自动退栈。
- 退栈后指向函数fun执行后的返回地址，然后执行ret指令
 - 该指令将esp此时的值反馈给eip，eip则可以找到main中的下一条指令，继续执行





一个简单的例子

- eip找到正确的后续执行指令之后，即将返回值赋值给正确的局部变量，即m。如右图所示



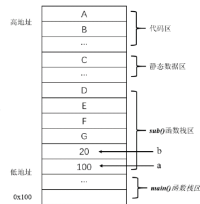
- 请考虑以下C语言代码。我们假设BP是指向栈底的指针，SP是指向栈顶的指针，C语言栈帧的结构如右图所示。本题中，栈帧布局地址增长遵循课本规定，栈是由低地址向高地址增长，其他部分则是由高地址向低地址增长。

- (1) A: _____
- (2) B: _____
- (3) C: _____
- (4) D: _____
- (5) E: _____
- (6) F: _____
- (7) G: _____
- (8) SP: _____
- (9) BP: _____

```
(1) int sub (int a, int b);
(2) int m = 0;
(3) void main(){
(4) int i = 100;
(5) int j = 20;
(6) m = sub(i, j);
(7) printf("m=%d", m)
(8) }
(9)
(10) int sub (int a, int b){
(11) int res = 0;
(12) res = a - b;
(13) return res;
(14) }
```

假设程序运行完第 12 行，即 *res* 计算完毕时，请在如下的栈帧结构图中，分别写出此时 A,B,C,D,E,F,G 对应位置的内容，若对应位置可以推断具体值（值可以为 null），请写明对应变量名及其值，并在图中通过箭头表明 *sub()* 函数的 SP 和 BP 的位置（假设 SP 和 BP 的位置在 *res* 被定义时已经确定）。

局部变量	高地址
BP的值	
返回地址	
返回值	
实参	低地址



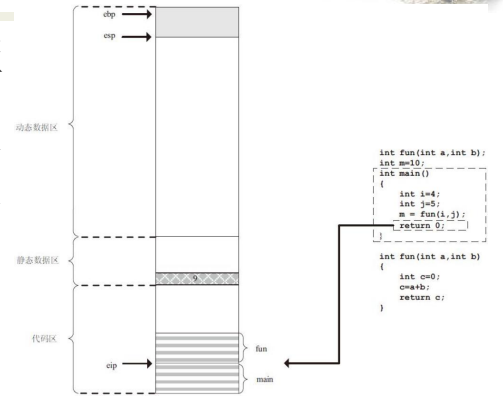
堆管理

- 堆空间
 - 用于存放生命周期不确定、或生存到被明确删除为止的数据对象
 - 例如：new生成的对象可以生存到被delete为止
 - malloc申请的空间生存到被free为止
- 存储管理器
 - 分配/回收堆区空间的子系统
 - 分配：它尽可能使用堆中的空闲空间来满足分配请求。如没有请求大小的空间块可供分配，试图从操作系统中获得连续的虚拟内存来增加堆区的存储空间。如无可用空间，通知应用
 - 回收：通常不会将内存返回给操作系统，而是用于后续数据对象分配
 - 根据语言而定
 - C、C++需要手动回收空间
 - Java可以自动回收空间（垃圾收集）



一个简单的例子

- 同理，在执行完main相关的ret指令之后，也需要对main函数的栈进行清除，清楚后的结果如右图所示
- 至此，整个程序执行完毕



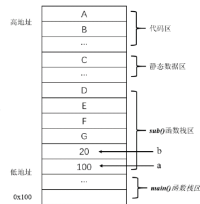
- 请考虑以下C语言代码。我们假设BP是指向栈底的指针，SP是指向栈顶的指针，C语言栈帧的结构如右图所示。本题中，栈帧布局地址增长遵循课本规定，栈是由低地址向高地址增长，其他部分则是由高地址向低地址增长。

- (1) A: 代码 int i = 100
- (2) B: 代码 int j = 20
- (3) C: m = 0
- (4) D: 局部变量 res = 80
- (5) E: BP
- (6) F: 返回地址
- (7) G: 返回值 = null
- (8) SP: D 的上方
- (9) BP: a 下方

```
(1) int sub (int a, int b);
(2) int m = 0;
(3) void main(){
(4) int i = 100;
(5) int j = 20;
(6) m = sub(i, j);
(7) printf("m=%d", m)
(8) }
(9)
(10) int sub (int a, int b){
(11) int res = 0;
(12) res = a - b;
(13) return res;
(14) }
```

假设程序运行完第 12 行，即 *res* 计算完毕时，请在如下的栈帧结构图中，分别写出此时 A,B,C,D,E,F,G 对应位置的内容，若对应位置可以推断具体值（值可以为 null），请写明对应变量名及其值，并在图中通过箭头表明 *sub()* 函数的 SP 和 BP 的位置（假设 SP 和 BP 的位置在 *res* 被定义时已经确定）。

局部变量	高地地址
BP的值	
返回地址	
返回值	
实参	低地址



存储管理器

- 基本功能
 - 分配：为每个内存请求分配一段连续的、适当大小的堆空间
 - 从空闲的堆空间分配，如果失败，则从操作系统中获取内存、增加堆空间
 - 回收：把被回收的空间返回空闲空间缓冲池，以满足其他内存需求
- 评价存储管理器的特性
 - 空间效率：使程序需要的堆空间最小，即减小碎片
 - 通过使存储碎片达到最少而得到
 - 程序效率：充分运用内存系统的层次，提高效率
 - 关注对象在存储中的放置方法，更好地利用空间，并且有希望使程序运行得更快
 - 低开销：使分配/收回内存的操作尽可能高效



存储管理器

- 处理器通常具有少量寄存器，寄存器中的内容由软件控制
 - 有一层或多层高速缓存，通常用静态RAM制造，主存通常用动态RAM实现
 - 在一次内存访问中，首先在最近的存储中寻找数据，如找不到，到下一层寻找
- 数据以连续存储块的方式进行传输
 - 在主存和高速缓存之间的数据是按照被称为高速缓存线的块进行传输的
 - 高速缓存线的长度通常在32-256字节之间
 - 在虚拟内存和主内存之间的数据以“页”的内存块进行传输的大小通常在4-64KB之间

典型的大小		典型的访问时间
> 2GB	虚拟内存（磁盘）	3~15 ms
256MB~2GB	物理内存	100~150 ns
128KB~4MB	二级高速缓存	40~60 ns
16~64KB	一级高速缓存	5~10 ns
32个机器字	寄存器（处理器）	1 ns



堆空间分配方法

- Best-Fit
 - 总是将请求的内存分配在满足请求的最小的窗口中
 - 好处：可以将大的窗口保留下来，应对更大的请求
- First-Fit
 - 总是将对象放置在第一个能够容纳请求的窗口中
 - 放置对象时花费时间较少，但是总体性能较差
 - 但是first-fit的分配方法通常具有较好的数据局部性
 - 同一时间段内的生成的对象经常被分配在连续的空间内



可达性

- 直观地讲，可达性就是指一个存储块可以被程序访问到
- 根集：不需要指针解引用就可以直接访问的数据
 - Java：静态成员、栈中变量
- 可达性
 - 根集的成员都是可达的
 - 对于任意一个对象，如果指向它的一个指针被保存在可达对象的某字段中、或数组元素中，那么这个对象也是可达的
- 性质
 - 一旦一个对象变得不可达，它就不会再变成可达的



堆空间的碎片问题

- 随着程序分配/回收内存，堆区逐渐被割裂成为若干空闲存储块（窗口，hole）和已用存储块的交错
- 分配一块内存时，通常是把一个窗口的一部分分配出去，其余部分成为更小的块
- 回收时，被释放的存储块被放回缓冲池。通常要把连续的窗口接合成为更大的窗口



垃圾回收

- 垃圾
 - 狭义：不能被引用（不可达）的数据
 - 广义：不需要再被引用的数据
 - 基于类型信息，可知道该对象有多大，及该对象的哪些分量包含指向其他对象的引用
 - 假定对对象的引用总是指向该对象的起始位置
 - 把一个用户程序称为增变者，它会修改堆中的对象集合
 - 增变者从存储管理器处获取空间，创建对象，还可引入和消除对已有对象的引用。当增变者不能“到达”某些对象时，这些对象就变成了“垃圾”
- 垃圾回收：自动回收不可达数据的机制，解除了程序员的负担
- 使用的语言
 - Java、Perl、ML、Modula-3、Prolog、Smalltalk



垃圾回收方法分类

- 跟踪相关操作，捕获对象变得不可达的时刻，回收对象占用的空间
- 在需要时，标记出所有可达对象、回收其它对象



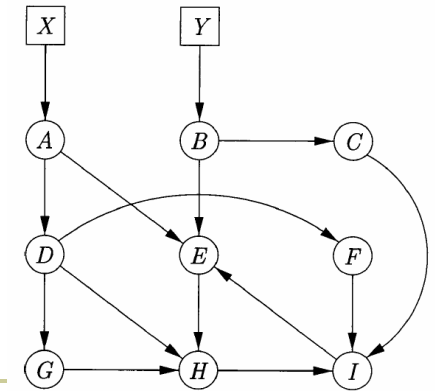
基于引用计数的垃圾回收器

- 每个对象有一个用于存放引用计数的字段，并按照如下方式维护
 - 对象分配：引用计数设为1
 - 参数传递：引用计数加1
 - 引用赋值： $u=v$ ：u指向的对象引用减1、v指向的对象引用加1
 - 过程返回：局部变量指向对象的引用计数减1
- 如果一个对象的引用计数为0，在删除对象之前，此对象中各个指针所指对象的引用计数减1
- 回收器有缺陷，可能引起内存泄漏
 - 不能回收不可达的循环数据结构
- 开销较大、但是不会引起停顿

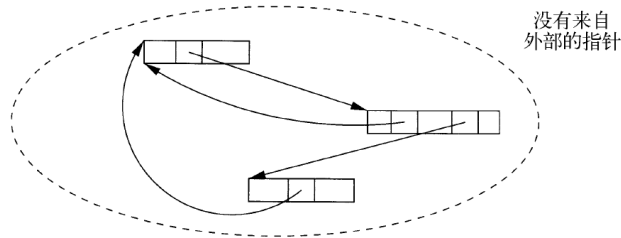


引用计数的例子

- 考虑如下操作：
 - $y=x$
 - y是当前函数f的局部变量，且f返回
- 修改计数后总是先考虑是否释放
- 释放一个对象之前总是先处理对象内部的指针



引用计数的例子



基于跟踪的垃圾回收

- 功能：
 - 不在垃圾产生时就回收，而是周期性运行
 - 寻找不可达对象并收回它们的空间
 - 通常是空闲空间被耗尽或空闲空间数量低于某个阈值时启动
- 标记-清扫式垃圾回收
- 标记-清扫式垃圾回收的优化
- 标记并压缩垃圾回收



标记-清扫式垃圾回收

- 一种直接的、全面停顿的算法
- 分成两个阶段
 - 标记：从根集开始，跟踪并标记出所有可达对象
 - 清扫：遍历整个堆区，释放不可达对象
 - 找出所有不可达的对象，将它们放入空闲空间列表
- 如果我们把数据对象看作顶点，引用看作有向边，那么标记的过程实际上是从根集开始的图遍历的过程



标记-清扫式垃圾回收

- 该算法主要包括两步
 - mark，从 root 开始进行树遍历，每个访问的对象标注为「使用中」
 - sweep，扫描整个内存区域，对于标注为「使用中」的对象去掉该标志，对于没有该标注的对象直接回收掉



标记-清扫式垃圾回收

- 输入：一个由对象组成的根集，一个堆和一个被称为Free的包含了堆中所有未分配存储块的空闲空间列表。所有空间块都用边界标记进行标识，指明它们的空闲 / 已用状态和大小。
- 输出：在删除了所有垃圾之后的经过修改的Free列表



标记-清扫式垃圾回收

- 该算法的缺点有
 - 在进行 GC 期间，整个系统会被挂起（暂停，Stop-the-world），所以在一些实现中，会采用各种措施来减少这个暂停时间
 - heap 容易出现碎片。实现中一般会进行 move 或 compact。（需要说明一点，所有 heap 回收机制都会这个问题）
 - 在 GC 工作一段时间后，heap 中连续地址上存在 age 不同的对象，这非常不利于引用的本地化（locality of reference）
 - 回收时间与 heap 大小成正比

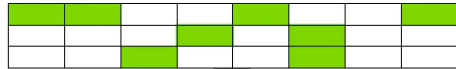


标记-清扫垃圾回收算法

回收前状态：



回收后状态：



存活对象

可回收

未使用



标记-清扫垃圾回收算法

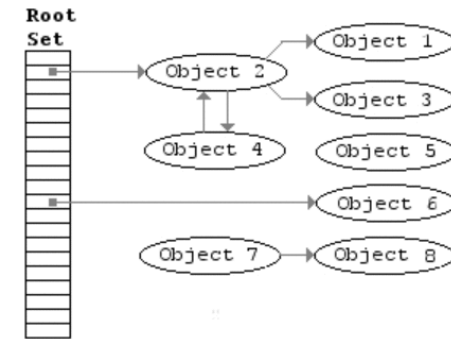
```

/* 标记阶段 */
1) 把被根集引用的每个对象的 reached 位设置为 1，并把它加入到 Unscanned 列表中；
2) while (Unscanned ≠ ∅) {
3)   从 Unscanned 列表中删除某个对象 o；
4)   for (在 o 中引用的每个对象 o' ) {
5)     if (o' 尚未被访问到，即它的 reached 位为 0) {
6)       将 o' 的 reached 位设置为 1；
7)       将 o' 放到 Unscanned 中；
8)     }
9)   }
10) }
/* 清扫阶段 */
8) Free = ∅；
9) for (堆区中的每个内存块 o) {
10)   if (o 未被访问到，即它的 reached 位为 0) 将 o 加入到 Free 中；
11)   else 将 o 的 reached 位设置为 0；
12) }
    
```

因为语言是强类型的，所以垃圾回收机制可以知道每个数据对象的类型，以及这个对象有哪些字段是指针

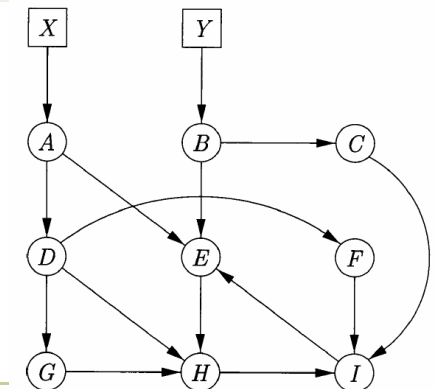


标记-清扫垃圾回收算法



例子

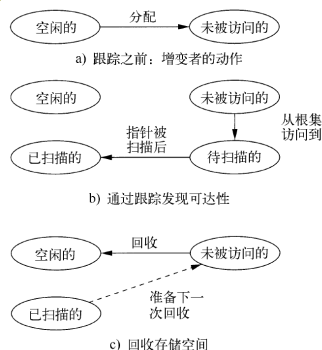
- 假设x是全局变量，y是当前函数活动的局部变量
- 当前活动返回之后，进行标记清扫
 - A, D, E, F, I, G, H
 - B, C不可达





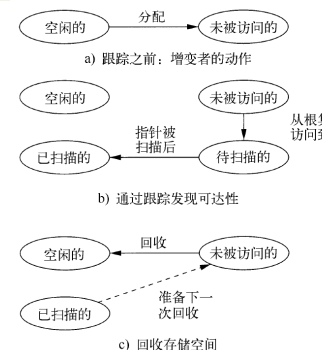
基本抽象分类

- 所有基于跟踪的算法都计算可达对象集合，然后取这个集合的补集
- 使用方式如下：
 - 程序（增变者）运行并发出分配请求
 - 垃圾回收器通过跟踪揭示可达性
 - 垃圾回收器收回不可达对象的存储空间



基本抽象分类

- 每个存储块处于四种状态之一
 - 空闲：可被分配，不存放可达对象
 - 未被访问：默认不可达，未确定可达性前，都标记为未被访问
 - 待扫描：存储块可达，但块中指针还没被扫描，则处于待扫描状态
 - 已扫描：只包含指向其他已扫描或待扫描对象的引用
- 对存储块的操作会改变存储块的状态
 - 应用程序分配
 - 垃圾回收器访问、扫描
 - 收回



标记-清扫垃圾回收算法的优化

- 基本算法需要扫描整个堆，代价很大
- 优化
 - 用一个列表记录所有已经分配的对象
 - 不可达对象等于已分配对象减去可达对象
 - 已分配对象和可达对象之间的差集
- 好处
 - 只需要扫描这个列表就可以完成清扫
- 坏处
 - 需要维护这个列表



压缩并标记垃圾回收

- 存在多种进行重新定位的回收器，不同之处在于它们是在本地进行重新定位还是在重新定位前预留了空间
 - 本节介绍的压缩并标记垃圾回收器在本地压缩对象，属于本地重新定位
 - 拷贝回收器，把对象从内存的一个区域移到另一个区域
 - 保留额外的空间用于重新定位可使得一发现可达对象就立刻移动它
- 对可达对象进行重定位可以消除存储碎片
 - 把可达对象移动到堆区的一端，另一端则是空闲空间
 - 空闲空间合并成单一块，分配内存时高效率
 - 可达对象占用的空间要大大小于空闲空间，标记出所有“窗口”后可以将所有可达对象重新定位到堆区的一端，使堆区的所有空闲空间成为一个块



压缩并标记垃圾回收

- 整个过程分成三个步骤
 - 标记阶段，参考之前的标记过程
 - 计算新位置
 - 算法扫描堆区中的已分配内存段，并为每个可达对象计算新的地址
 - 新地址从堆的最低端开始分配，可达对象之间没有空闲存储窗口
 - 每个对象的新地址记录在一个名为NewLocation的结构中
 - 移动并设置新的引用
 - 算法将对象拷贝到它们的新地址，更新对象中的所有引用
 - 使之指向新地址，新地址可在NewLocation中找到



压缩并标记垃圾回收

```

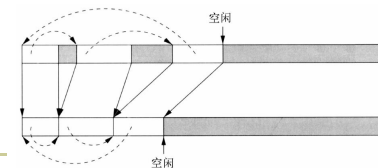
/* 标记 */
1)  Unscanned = 根集引用的对象的集合;
2)  while (Unscanned != 0) {
3)    从 Unscanned 中移除对象 o;
4)    for (在 o 中引用的每个对象 o') {
5)      if (o' 是未访问的) {
6)        将 o' 标记为已访问的;
7)        将 o' 加入到列表 Unscanned 中;
      }
    }
  }
/* 计算新的位置 */
8)  free = 堆区的开始位置;
9)  for (从低端开始, 遍历堆区中的每个存储块 o) {
10)   if (o 是已访问的) {
11)     NewLocation(o) = free;
12)     free = free + sizeof(o);
   }
 }

```

```

/* 重新设置引用目标并移动已被访问的对象 */
13) for (从低端开始, 堆区中的每个存储块 o) {
14)   if (o 是已访问的) {
15)     for (o 中的每个引用 o.r) {
16)       o.r = NewLocation(o.r);
17)     }
18)   }
19)   r = NewLocation(r);
}

```



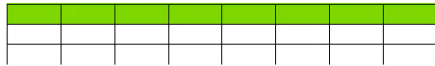


压缩并标记垃圾回收

回收前状态：



回收后状态：



存活对象

可回收

未使用



拷贝回收器

- 拷贝回收算法虽然能解决内存碎片问题，但是需要多次遍历heap空间，这会导致较大性能损耗
- 拷贝回收算法采用空间换时间的方式来提升性能
 - 输入：一个由对象组成的根集，一个包含了From半空间和To半空间的堆区，其中From半空间包含了已分配对象，To半空间全部是空闲的
 - 输出：最后，To半空间保存已分配的对象。free指针指明了To半空间中剩余空闲空间的开始位置。From半空间此时全部空闲
 - 方法：在From半空间找出可达对象，访问它们时立刻把它们拷贝到To半空间
- 这类GC并不会真正去“回收”不可到达对象，而是会把所有可到达对象移动到一个区域，heap中剩余的空间就是可用的了（因为这里面都是垃圾）



拷贝回收器

- 输入：一个由对象组成的根集，一个包含了From半空间和To半空间的堆区，其中From半空间包含了已分配对象，To半空间全部是空闲的
- 输出：最后，To半空间保存已分配的对象。free指针指明了To半空间中剩余空闲空间的开始位置。From半空间此时全部空闲
- 方法：
 - 算法在From半空间找出可达对象，访问它们时立刻把它们拷贝到To半空间



拷贝回收器

- 预先保留了可将对象移入的空间，解除了跟踪和发现空闲空间间的依赖关系
- 堆空间被分为两个半空间
 - 增变者在某个半空间内分配存储，当充满这个半空间时，开始垃圾回收
 - 回收时，可达对象被拷贝到另一个半空间
 - 回收完成后，两个半空间角色对调
 - 增变者可继续运行，并在半空间B中分配对象
 - 下一轮垃圾回收将把可达对象移动到A



拷贝回收器

- 典型的代表半空间拷贝回收器（semispace collector）。其工作过程是这样的
 - heap被分成2份相邻的空间（semispace）：fromspace与tospace
 - 在程序运行时，只有fromspace会被使用（分配新对象）
 - 在fromspace没有足够空间容纳新对象时，程序会被挂起，然后把fromspace的可到达对象拷贝到tospace
 - 在拷贝完成时，之前的2个空间交换身份，tospace成了新一轮的fromspace



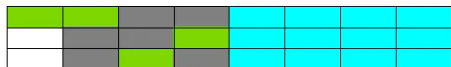
拷贝回收器

```

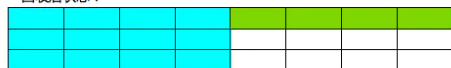
1) CopyingCollector () {
2)   for (From空间中的所有对象o) NewLocation(o) = NULL;
3)   unscanned = free = To空间的开始地址;
4)   for (根集中的每个引用r)
5)     将r替换为 LookupNewLocations(r);
6)   while (unscanned != free) {
7)     o = 在unscanned所指位置上的对象;
8)     for (o中的每个引用 o.r)
9)       o.r = LookupNewLocation(o.r);
10)    unscanned = unscanned + sizeof(o);
11)  }
12)  /* 如果一个对象已经被移动过了，查找这个对象的新位置 */
13)  /* 否则将对象设置为待扫描状态 */
14)  LookupNewLocation(o) {
15)    if (NewLocation(o) = NULL) {
16)      NewLocation(o) = free;
17)      free = free + sizeof(o);
18)      将对象o拷贝到 NewLocation(o);
19)    }
20)    return NewLocation(o);
21)  }
22) }
    
```



回收前状态：



回收后状态：



存活对象

可回收

未使用

保留区域



■ 采用该策略的基本原因：

- 因为较年轻的世代往往包含较多的垃圾，也就更频繁地被回收
- 编程实践中，极少出现较老世代对较新世代的引用。因此，我们只需要记录该种情况，并定期采用其他策略进行处理即可



2. 从 A 指向 B 的指针被删除时，引用计数会发生哪些变化，画出对应的指针被删除后的对象网络。

X

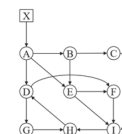


- 针对很多变量存在时间较短的特点设计
- 基本思想：将堆区进行切割成块并标号，垃圾回收技术每次在较低标号的块上进行
- 具体执行方案：
 - 新对象在编号较低的堆块上创建
 - 每次在0号堆块被填满时刻，就对0号堆块进行垃圾回收；回收完毕后，将i号堆块往后移动到i+1号堆块中
 - 维护一个记忆集，用以存储较高号的堆块对较低号堆块中对象的引用



七、简答题（共1题，共10分）

现有如下图所示的对象网络：



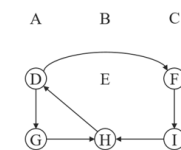
得分	
评分人	

1. 从 X 指向 A 的指针被删除时，对象的引用计数会发生哪些变化，画出对应的指针被删除后的对象网络（保留下来的对象请在字母外画上实线圈，被删除对象不作处理，保留的指针画上箭头，被删除的指针不作处理）。

X



X



L₁

X

